INDEX

NAME Balaji.S STD. SEC. ROLL NO 2407a069 SUB.

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
1.	26/07	Study of network commands	8	27/8/26
2	28/07/26	study of different types of network cables.	9	4/8/26
3)	3/8/26	CISCO PACKET TRACER	9	18/8/26
4	10/8/26	Setup and configure a LAN using switch and ethernet cables	9	18/8
5	18/8/26	WIRESHARK	9	18/8
6	7/9/26	Error detection & correction using hamming code	10	7/9/26
7	22/9/26	NMAP to discover live hosts using Nmap scans on the tryHackme platform	9	22/9
8	28/9/26	End to End communication at transport layer	9	28/9
9	28/9/26	Sliding window protocol	9	28/9/26

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
10	29/9/26	Implementation of ~~socket~~ Subnetting in cisco packet tracer simulator		
11.	5/10/26	Internetworking with Routers with cisco packet tracer simulator		
12.	13/10/26	Routing at network layer		
13.	13/10/26	Implement your own ping program		
14.	13/10/26	Packet sniffing using Raw sockets.		

nslookup:

(name server lookup) is a tool used to perform DNS lookups in Linux. It is used to display DNS details, such as the IP address of a particular computer, the MX records for a domain or the NS servers of a domain. nslookup can operate in two modes: interactive and non-interactive. eg :- nslookup www.google.com.

Pathping:

Pathping is unique to window's and is basically a combination of the ping and Tracert commands. pathping traces the route to the destinating address then launches as 0s second test of each router along the way gathering statistics on the rate of data loss along each hop

Ping:

(Packet Internet Groper) command is the best way to test connectivity between two nodes. Ping use ICMP (Internet control message Protocol) to communicate to other devices

1) #ping hostname (Ping localhost)
2) # Ping IP address (Ping 4.2.2.2)
3) # ping fully qualified domain name (Ping www.facebook.com)

Route:

route command is used to show / manipulate the IP routing table. It is primarily used to setup static routes to specific host or networks via an interface.

Ex no: 1

Aim: - To Study of various Network commands used in linux and windows.

arp -a :

ARP is short form of address resolution protocol. it will show the IP address of your computer along with the IP address and MAC address of your router

hostname:

This is the simplest of all TCP/IP commands it simply display the name of your computer

ipconfig /all

This command displays detailed configuration information about your TCP/aIP connection including Router, Gateway, DNS, DHCP, and type of Ethernet adapter in your system.

nbtstat -a:

This commands helps solve problems with Net BIOS name resolution. (Nbt stands for Net BIOS over TCP/IP)

netstat:

(Network statistics) netstat displays a variety of statistics about a computers active TCP/IP connections both incoming and outgoing as well as viewing routing tables, interface statistic etc.

Eg: netstat -r

Some important linux networking commands

1) IP

The ip commands is one of the basic commands every administrator will need in daily work. from setting up new systems and assigning IPs to troubleshooting existing system. The ip command can show address information, manipulate routing. plus display network various devices, interfaces and tunnels.

IP <OPTIONS><OBJECT><COMMANDS>

a) To show the IP address to an interface on your server:

a) [root@server ~] # IP address show

b) To assign a IP to an interface, for example enps03:

a. [root@server ~] # IP address add 192.168.1.254/24 dev enps03

c) To delete an IP on an interface:

a. [root@server ~] # IP address del 192.168.1.254/24 dev enps03

d) Alter the status of the interface by bringing the interface eth0

online: [root@server ~] # IP link set eth0 up

e) Alter the status of the interface by bringing the interface eth0

offline: [root@server ~] # IP link set eth0 down

f) Alter the status of the interface by enabling promiscuous mode for

eth0: [root@server ~] # ip link set eth0 promisc on

g) Add a default route (for all address) via the local gateway 192.168.1.254 that can be reached on device eth0:

[root@server ~] # IP address route add default via 192.168.1.254 dev eth0.

h. Add a route to 192.168.1.0/24 via the gateway at 192.168.1.254:

[root@server ~]# ip route add 192.168.1.0/24 via 192.168.1.254.

i. Add a route for 192.168.1.0/24 via the gateway of that can be reached on device.

Utho: [root@server ~] # ip route add 192.168.1.0/24 dev utho

j. Delete the route for 192.168.1.0/24 via the gateway at 192.168.12.254:

[root@server ~]# ip route delete 192.168.1.0/24 via 192.168.1.254

k. Display the route taken for IP 10.10.1.4:

[root@server ~]# IP route get 10.10.14

2. ~~ifconfig~~

The ifconfig command was is a staple in many sysadmin's tool belt for configuring and troubleshooting networks. it has since been replaced by the IP command discussed above.

3. ~~mtr~~

MTR (Matt's traceroute) is a program with a command-line interface that serves as a network diagnostic and troubleshooting tool This command combines the functionality of the ping and traceroute commands. Just like a traceroute commands. Just like a traceroute the mtr command will show the route from a computer to specified host mtr provides a lot of statistics about each hop - such as response

time and percentage. with the mtr command you will get more information about the route and the also the see problematic devices along the way if you see a sudden increase in response time or the syntax of the command is a follows,

mtr <options> hostname/IP

a) The basic mtr command show you the statistics including each hop (hostnames) with time and loss

o/o [root@server ~] # mtr google.com

b) show numeric IP address (if you use -g you will get IP address (numbers) instead of hostname):

[root@server ~] # mtr -g google.com

c) show the numeric IP addresses and hostnames too: [root@server ~] # mtr -b google.com.

d) set the number of pings that you want to send: [root@server ~] # mtr -c 10 google.com

4. tcpdump

The tcpdump command is designed for capturing and displaying packets.

you can install tcpdump with the command below:

[root@server ~] # dnf install -y tcpdump.

Before starting any capture you need to know which interface can use you will need to use sudo or have root access in this case [root@server ~] # tcpdump -D

if you want to capture traffic on etho you can initiate that with tcpdump -i etho example output: [root@server ~] # tcpdump -i etho

[root@server ~] # tcpdump -i eth0 -c10

capture traffic to and from one host
you can filter out traffic coming from a specific host. For example, to find traffic coming from and going to 8.8.8.8 use the command.
[root@server ~] #tcpdump -i eth0 -c 10 host 8.8.8.8

For traffic coming from 8.8.8.8 use:
[root@server ~] # tcpdump -i eth0 src host 8.8.8.8

For outbound traffic going to 8.8.8.8 use
[root@server ~] # tcpdump -i eth0 dst host 8.8.8.8

capture traffic to and from a network
you can also capture traffic to and from a specific network using the command below
[root@server ~]# tcpdump -i eth0 net 10.10.0 mask 255.255.255.0
or:
[root@server ~] # tcpdump -i eth0 net 10.1.0.0/24

capture traffic to and from port numbers
capture only DNS port 53 traffic
[root@server ~] # tcpdump -i eth0 port 53
For a specific host
[root@server ~] # tcpdump -i eth0 host 8.8.8.8 and port 53
To capture only HTTPS traffic
[root@server ~] # tcpdump -i eth0 -c 10 host www.google.com and port 443

To capture all port except port 80 and 25
[root@server ~] # tcpdump -i eth0 port not 53 and not 25

Ping

Ping is a network utility used to check connectivity between two devices on a network. It sends ICMP Echo Request message to a destination host and waits for echo Reply message.

eg: Ping google.com

Ping 10 times

Ping -c 10 goggle.com

- Reply Received connection is working properly.
- Destination host unreachable: No route to the destination host
- Request Timed out: No response received from the host
- unknown host: Hostname is incorrect or cannot be resolved.

A good ping result should have low latency and 0% packet loss.

Result

The above both the windows and linux commands are executed and output are completed successfully.

27/8/21

Aim: Study of different types of Network cables

a) understand different types of Network cable

Different types of cables in network cable.

1) unshielded Twisted Pair (UTP) cable.
2) shielded Twisted Pair (STP) cable.
3) coaxial cable
4) Fibre optic cable.

Cable type	category	Maximum Data Transmission	Adv / disadvantages	Application/ Use.
UTP	category 3	10 bPS	**Advantages** * cheaper in cost * Easy to install as they have a smaller overall diameter **Disadvantages** * More prone to (EMI) Electromagnetic interfaca and noise	10Base -T Ethernet
	category 5	up to 100 MbPS		Fast Ethernet gigabit Ethernet
	category 5e	1Gb PS		Fast Ethernet gigabit Ethernet
STP	category 6, 6a	10GbPS	**Advantages** * shielded * Faster than UTP * Less Susceptible to noise and interference **Disadvantages** * Expensive * Greater installation effort	Gigabit Ethernet 10G Ethernet (55 m) widely used in data centers
SSTP	category 7	10Gb PS		Gigabit Ethernet 10G Ethernet (100 m)

axial cable	RG-6 RG-59 RG-11	10-100Mbps	**Advantage** * high bandwidth * Immune to interface * low loss bandwidt * Versatile **Disadvantage.** * limited distance * cost * size is bulky	speed e.g. is 500m Television network High speed internet connections.
fiber optics cable	single mode Multi mode	100Gbps	**Advantage** * high speed * High bandwidth * High security * loong distance **Disadvantage.** * Expensive * Requires skilled installers	* Maximum distance of fibre optics cable is around 100 meters.

Result:

Thus, the different types of network cables are studied succesfully.

Ex:3 Cisco Packet Tracer. 3/08/2024

Aim:

To study cisco packet tracer tool installation and user interface overview.

Introduction:

A simulator, as the name suggests, simulate network devices and its environment packet tracer is an exciting network design, simulation and modelling tool.

* It allows you to model complex systems without the need for dedicated equipment
* It helps you to practice your network configuration and trouble shooting skills via computer or an Android. or ios based mobile device
* It is available for both the linux and windows desktop environments.
* protocols in packet tracer are coded to work and behave in the same way as they would on real hardware.

Analyze the behaviour of network devices using cisco packet Tracer simulator.

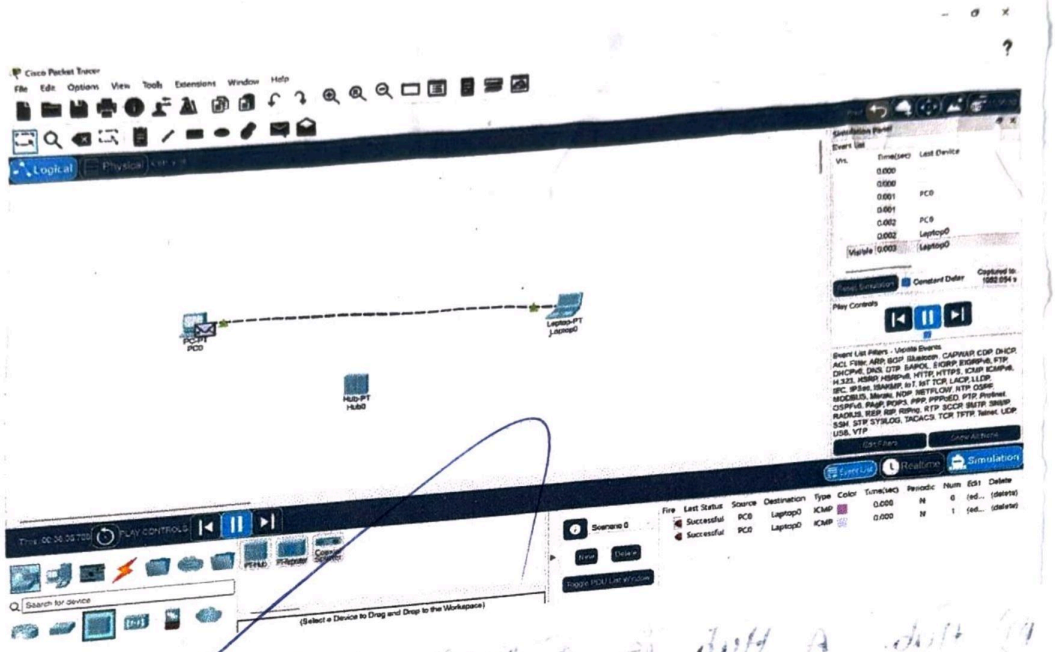
i) Pc to pc
ii) switch
iii) Hub.

i) Peer to Peer

A Peer to Peer network is a network in which computers communicate directly with each other. without requiring a dedicated central server.

Ex:

Two PC's connected directly using a cable and communicating with each other.

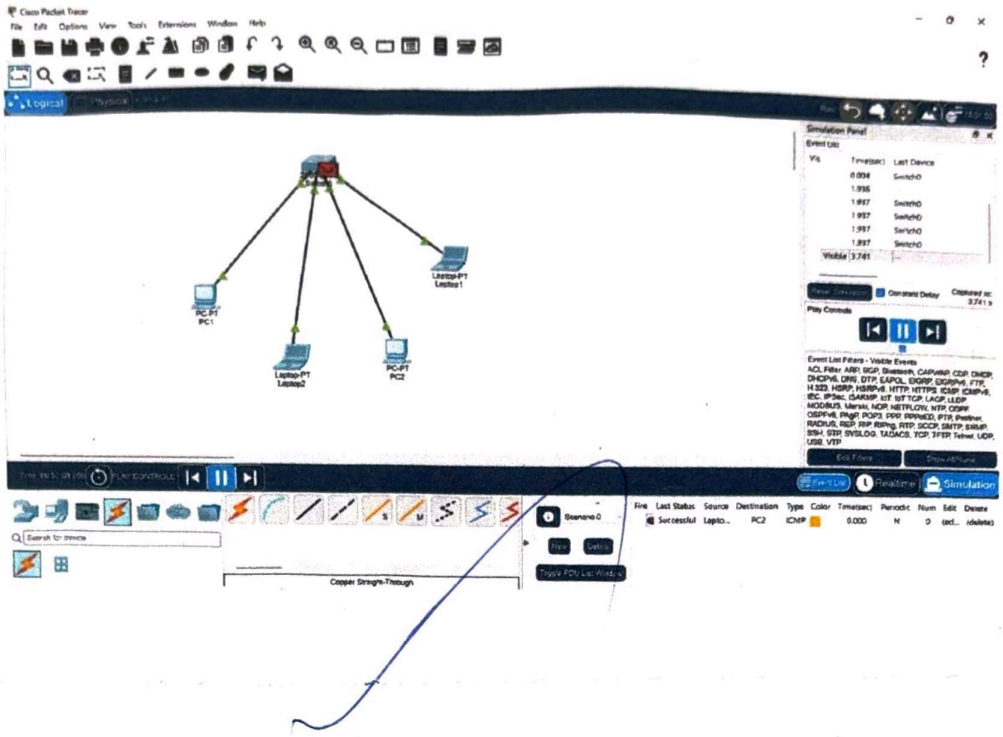


ii) switch :-

A switch is a networking device used to connect multiple devices in a network and forward data only to the intended destination device

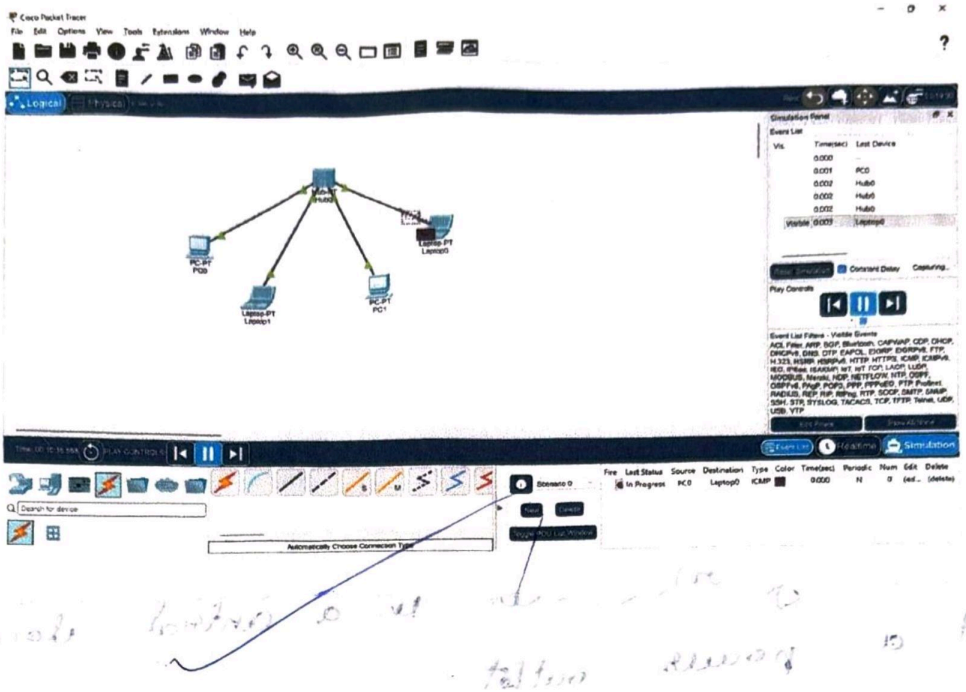
Eg: Pc1 -> switch -> Pc3

switch knows where Pc 3 is and forwards the frame only towards Pc 3 switch only works in the data link layer (layer 2) of the OSI model.



iii) Hub: A Hub is a basic networking device used to connect multiple computers in a network it receives data from one device & broadcasts it to all connected devices Hub works mainly at physical ~~to~~ layer (layer 1) of the OSI model

Eg: If PC1 sends data to PC3 through a hub the hub sends that data to PC2 , PC3 , PC4 etc.



Result : Thus the cisco packet tool is installed successfully & behaviour of network devices using cisco is studied.

Ex 4 Steup and configure a lan using switch and Ethernet cables.

what is Lan ?

Lan stands for local area Network which is a group of computer and connected devices linked togther in a small physical location like a home, school or office sending

How to set up LAN:

Setp :1 connect the Hardware

-> Place your router in a central location near a power outlet

-> connect your modem to the router's LAN or Internet port using an Ethernet cable.

-> Plug the router into power and wait for it to turn on.

step 2: connect your devices.

-> wired setup are end of an ethernet cable into your computer network port and the other end into one of the numbered lan pross on the router.

stop: 3 configure the router

-> Type your IP address (192.168.1.1 or 192.168.0.1) into the address bar and press enter.

Step 4: Check the configuration between switch and other machine. by using ping command in the prompt the device.

Step 5: Tag to access the shared folder from other computer of the networks.

Result:

Workstations LAN using switch and ethernet cable are accepted successfully from cisco packet tracer.

10/8/24

3) create a filter to display only HTTP packets and inspect the packets procedure.

* select local area connection in wireshark
* go to capture option
* select stop capture automatically after 100 packets
* Then click save capture
* search HTTP packets in search bar.

4) create a file to display only DNS packets and provide the flow graph

Procedure:

* go to capture option
* select stop capture automatically after 100 packets
* Then click start capture
* search DNS packets in search bar
* To see the flow graph click statistics flow graph.

5) create a filter to display only DHCP packets and inspects the packets.

* select local connection in wireshark.
* go to capture option.
* select stop capture automatically after 100 packets.
* Then click start capture.
* search DHCP packets in search bar
* save the packets.

Exp:5 WIRE SHARK

Aim

Experiments on packet capture wireless to filter capture view, Packets in wire shark tool. capture 100 packets from the ethernet IEEE 802.3 LAN interface and save it

Procedure:

* select local area connection in wire shark.
* To capture option
* select stop capture automatically after 100 packets.
* then click, start capture.
* search Top packets in search bar.
* To see flow graph then click statistics flow graph
* save the packets.

i) create a filter to display only TCP / UDP Packets.

* select LAN connection in wireshark.
* to capture option.
* select stop capture automatically after, 100 packets
* Save the packets.

2) create a filter to display only ARP Packets and packets the packetes procedure.

* Go to captures option.
* Then click start capture
* Search ARP Packets in search bar
* Save the Packets

Result:

We learnt about packet capture tool wireshark and executed successfully.

18/8/21

Exp:6 Error detection & correction using Hamming code

Aim:

write a program to implement error detection and correction using hamming code. concept. Make a test run to input data stream and verify error correction feature.

sender:

* Input to sender file should be a text. of any length program should be convert the text to binary
* Apply hamming code concept on the binary data and add redundant bits to it.
* save this output in a file called channel.

code:

```
data = input ("Enter 4 bits")
d1, d0, d3, d4 = map (int, data)
P1 = d1 ^ d2 ^ d4
P2 = d1 ^ d3 ^ d4
P3 = d2 ^ d3 ^ d4
code = [P1, P2, d1, P4, d2, d3, d4]
print ("sender Hamming code :", "".join
(map (str, code)))
```

Receiver side

* Receive the hamming code from the sender
* Enter the received code.
* Calculate the parity check values.
* Find the error position using the parity
* Correct the error by changing the bit
* Display the corrected hamming code.

Code:

```
H = list(map(int, input("Enter received bits: ")))
H1 = H[0] ^ H[2] ^ H[4] ^ H[6]
H2 = H[1] ^ H[2] ^ H[5] ^ H[6]
H3 = H[3] ^ H[4] ^ H[5] ^ H[6]
error = H1*1 + H2*2 + H4*4
if error == 0:
    print("No Error")
else:
    print("Error at position:", error)
    H[error-1] = 1
print("corrected code:", "".join(map(str, received)))
```

Input:

Enter 4 data bits: 1011
Sender hamming code: 0110011
Received 7bits: 0010011

Output:

Error at Position: 2
Correct code: 0110011

Sample Input 2:

Enter 4 data bits : 0010

Sender hamming code : 0101010

Enter received 7 bits : 0101010

output:

No Error!

Result:

Error detection & Error correction using hamming code was executed successfully.

7/9/24

EXP 7. NMAP to Discover live hosts using Nmap scans on the try hackme platform.

Aim:

To Discover live hosts using NMAP scans (ARP, ICMP, TCP/UDP) on the Try back me platform Room. link: https://tryhackme.com/room/

Procedure:

Task: . introduction:

* ARP scan: This scan was ARP requests to discover live hosts.

* ICMP scan: This scan uses ICMP requests to identify live hosts.

* TCP/UDP Ping scan: This ~~scan~~ scan sends Packets To TCP Ports and UDP Ports to determine live hosts.

Task 2: Subnetworks:

Network segments - It is a collection of Computers linked together through a common "medium".

Subnetworks - it is usually the equivalent of one or more network segments linked together and configured to subtype the same router in an IP networks.

* subnets with /16
* subnets with /24

Task 3: Enumerating Targets.

We need to identify the targets we want to scan before we conduct any "action"

1) list
   - <Machine - IP> <scan.txt, nmap.org> <example.com>
   - will scan 3 IP addresses.

2) range
   - 10.11.12.15.20
   - will scan 6 IP address.
   nmap -sL <TARGETS>

Task 4: Discovering live Hosts.

1) ARP from link layer, commonly used for man-in-middle attack (ARP spoofing and ARP Poisoning).

2) ICMP from Network layer - commonly used in DDOS attacks.

3) TCP from Transport layer - syn flooding, TCP reset, TCP session hijacking attacks.

4) UDP from Transport layer - Also used in DDOS attacks.

Task: 5. NMAP Host Discovery using ARP.

1) When a non-privileged user tries to scan LAN (local Area Network), NMAP uses ARP requests.

2) to discover Internet hosts without port scanning - like system use nmap -sn targets

3) nmap -PR -sn <targets>

4) nmap -PR -sn <MACHINE IP> /24

5) nmap -PR -sn <Attack Box IP> /24

Task 6: nmap host discovery using ICMP

i) Add -PE option to utilize ICMP echo requests to discover live host.

2) Nmap -PE -sn -Target to check host is live

3) -PP option instructs nmap to use timestamp requests

nmap -PP -sn Target.

4) -Pm option can enable this scan to ICMP address mask requests.

nmap -Pm -sn Target.

Task 7: Nmap host discovery using TCP and UDP

1) Nmap to use TCP syn Ping use -PS flowed by the port number, range, list or a combination of this options.

-PS21, -PS25, Target ports 21, 22, 23, 24, 25,

2) nmap -PS -sn Target check the TCP port is open.

3) nmap -PR -sn Target to discover the online hosts on the target's subnet.

4) UDP Ping - sending a UDP packets to an open port is unlikely to result in a response

Task 8: using Reverse - DNS lookup

1) Nmap is to use reverse - DNS on target hosts.

2) -R option to query the DNS server even for offline hosts.

Task 9: for summary:

ARP scan, Icmp Echo scan, ICMP Time stamp scan, ICMP address mask scan, TCP syn ping scan, TCP ACK, Ping scan, UDP ping scan.

Result:

NMAP to discover live host using Platform was successfully cracked.

22/9/25 try hacking

Ex no: 8 End to End communication at Transport layer.

a)
Aim:

To implement echo client server using TCP/UDP Packets.

Procedure:

* Create a socket using python socket programming.
* Specify the IP address and port numbers.
* Bind the servers socket to the IP address and Port
* Make the server socket to the IP address and client connections.
* connect the client to the server.
* Send and receive messages between client and server.
* Display the received message.
* close the socket connection.

code:

TCP - server:

```
Import socket
server - socket. socket (socket . AF_INET, sock
                                    sock - STREAM)
server . bind ("127 . 0 . 0.1" , 5000))
server . listen (1)
print (" server is waiting for connection ...
conn, addr = server . accept ()
print ("client connected.
```

```
while true:
    data = conn.recv(1024).decode()
if not data:
    break
Print("client :", data)
conn.send(data.encode())
conn.close()
server.close()
```

TCP - Receiver (client)

```
Import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("Enter message:")
client.send(message.encode())
reply = client.recv(1024).decode()
Print("server:", reply)
client.close()
```

Terminal 1 - server

Python TCP - Server.py

Server is waiting for connection

client connected

client : Hello

Terminal 2 - client

Python tcp.client.py

Enter message : Hello

Server : Hello.

TCP echo is working

Result:

The TCP/UDP packets using echo client server socket program is executed successfully.

28/1/26

Experiment -

Sliding window Protocol

Aim:-

write a program to implement flow control at data link layer using sliding window protocol simulate the flow of

code

```
import time
window = int (input ("Enter window size: "))
message = input ("Enter Message : ")
start = 0
while start != len (message)
Print ("sender")
if end > len (message)
end = len (message)
# sender buffer
send - buffer = []
for i in range (start , end):
frame = (i , message[i])
sender - buffer . append (frame)
print ("Frame no : ", i, "Data : ", message[i])
Print ("in Sender Buffer:")
Print (sender . buffer)
# Delay
Print ("in writing your receiver . . . .")
time : sleep (1)
# Receiver.
Print ("Receiver")
Updated = start
```

```
frame_no = frame[0];
data = frame[1]
Print("Received Frame :", frame_no, "Data:", Data)
If frame_no = expected.
    expected = expected +1
else :
    error = true
    then
# receiver Buffer
if error:
    receiver_Buffer = ("NACH", expected)
    Print("\n Receiver Buffer :")
    Print( receiver_buffer)
else:
    ack = expected -1
    receiver_buffer = ("ACK", ack)
    Print("\n Receiver buffer :")
    Print("receiver_buffer")
# Sender reads ACK / NACK
    Print("\n SENDER CHECKING RESPONSE")
    response = receiver_buffer[0]
    number = receiver_buffer[1]
    If response == "ACK"
        Print("ACK received :", number)
        Start = number +1
    else:
        Print("NACK received :") number
        start = number
    Print("TRANSMISSION completed")
```

output:

Enter window size : 3

Enter message : Hello

Sender

Frame NO: 0 Data : H

Frame NO: 1 Data : e

Frame NO: 2 Data : l

Sender Buffer :

[(0, 'H'), (1, 'e'), (2, 'l')]

waiting for receiver

RECEIVER

Received Frame : 0 Data : H

Received Frame : 1 Data : e

Received Frame : 2 Data : l

('ACK', 2)

Sender checking Response

Sender

Frame no: 3 Data: l

Frame no: 4 Data: o

Sender Buffer:

[(3, 'l'), (4, 'o')]

waiting for receiver

RECEIVER

Received frame: 3 Data:

Received frame: 4 Data:

Received Buffer:

('ACK', 4)

Sender checking Response ACK

Received: 4

TRANSMISSION COMPLETED

Result:

sliding window Protocols are learned and executed successfully.

28/9/24

Exp-10 implementation of subnetting in cisco packet tracer simulator
29/9/24

Aim:

To implement subnetting using cisco packet tracer and verifying communication between the subnet-

Subnetting:

It is a process of dividing large network into smaller logical network called subnets it helps in efficient utilization of IP address, reduces unnessary network traffic & improve network mangement

Algorithm:

* Start cisco packet tracer
* place one router, two switches & four PCs on the work space
* connect 2pc to switch & both switches to the router
* Assign static IP address & subnet mask to all pcs
* configure IP address on the router interfaces connected to the two subnets
* configure default gateway for each pc
* check connectivity between pcs with same subnet using ping command
* Test communication between pcs belonging to different subnets
* Disconnect (or) disable the route interface & repeat communication test
* observer & record result

Procedure:

Step 1 : Design Network Topology

* connect PC 0 & PC 1 to switch 0
* connect PC 2 & PC 3 to switch 1

Step 2: Assign IP address

Device	Ipaddress	subnetmask	Default- Gate
PC0	192.168.1.2	255.255.255.0	192.168.1.1
PC1	192.168.1.3	255.255.255.0	192.168.1.1
PC2	192.168.1.4	255.255.255.0	192.168.1.1
PC3	192.168.1.5	255.255.255.0	192.168.1.1

Step 3 : Configure the Router.

* Open CLI
* Terminal

Enable
configuration terminal
interface gigabit Ethernet 0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit
interface gigabit Ethernet 0/1
ip address 192.168.2.1 255.255.255.0
no shutdown
Exit
end

Step 4 : Test communication between same subnet PC0 . Ping 192.168.1.3
check PC1 responds.

Step5: Test communication between different subnets pc0, ping 192.168.2.2

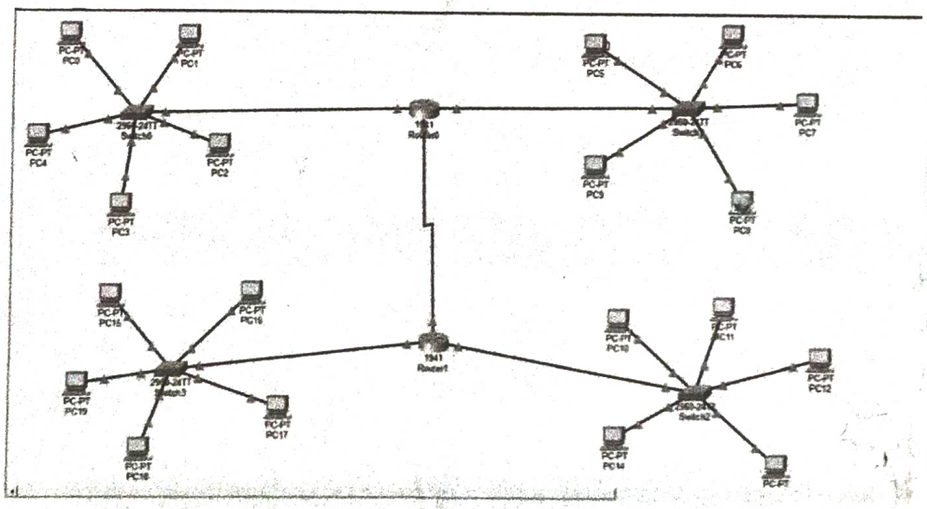
check whether pc_2 responds

Step 6: Test communication without Router

* Discount router links
* Attempt to ping pc_2 from pc0 again

Observation table

Source	Destination	Expected Result
pc0	pc_1	Successful
pc0	pc_2	Successful when router is configured & connect
pc0	pc_3	Unsuccessful when the router connection between subnets is disabled



Result:

Thus the Subnetting was Implemented Successfully using cisco packet tracer

Exp-11 - Internetworking with Router in CISCO
5/10/26 packet tracer simulator.

10(a) Internetworking using Routers

Aim:
To design and configure simple Internetwork using router in cisco packet tracer.

Algorithm:
* place router & two PCs in packet tracer
* connect PCs to router using straight through cables
* Assign IP address & subnet mask to PC & router interface
* configure router using CLI & enable them
* use ping to test connectivity.

Procedure:
* open packet tracer & create network topology
* configure router interface with IP address
* Assign IP address to the PC in different network
* enable router interface using no shutdown
* Test communication between PCs

Result:
The Internetwork was configured successfully & connectivity between PCs was tested using router

Exp-11 - Internetworking with Router in CISCO
5/10/26 packet tracer simulator.

10(a) Internetworking using Routers

Aim:
To design and configure simple Internetwork using router in cisco packet tracer.

Algorithm:
* place router & two PCs in packet tracer
* connect PCs to router using straight through cables
* Assign IP address & subnet mask to PC & router interface
* configure router using CLI & enable them
* use ping to test connectivity.

Procedure:
* open packet tracer & create network topology
* configure router interface with IP address
* Assign IP address to the PC in different network
* enable router interface using no shutdown
* Test communication between PCs

Result:
The Internetwork was configured successfully & connectivity between PCs was tested using router

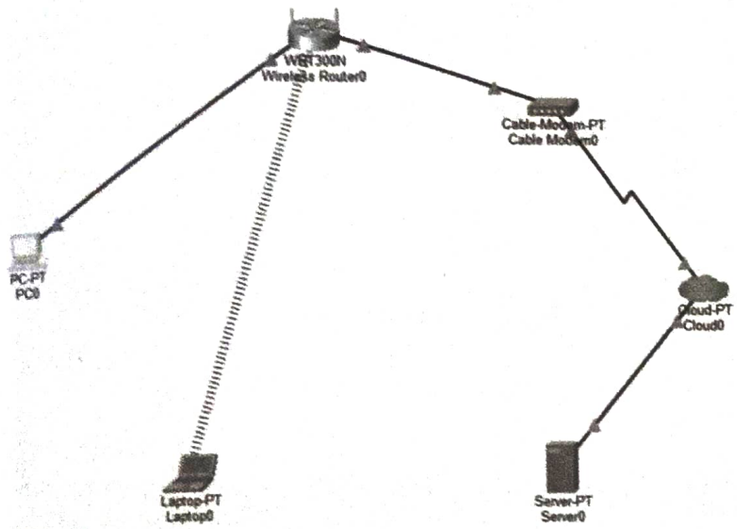
10 (b) Wireless Router, DHCP Server & Internet cloud

Aim:

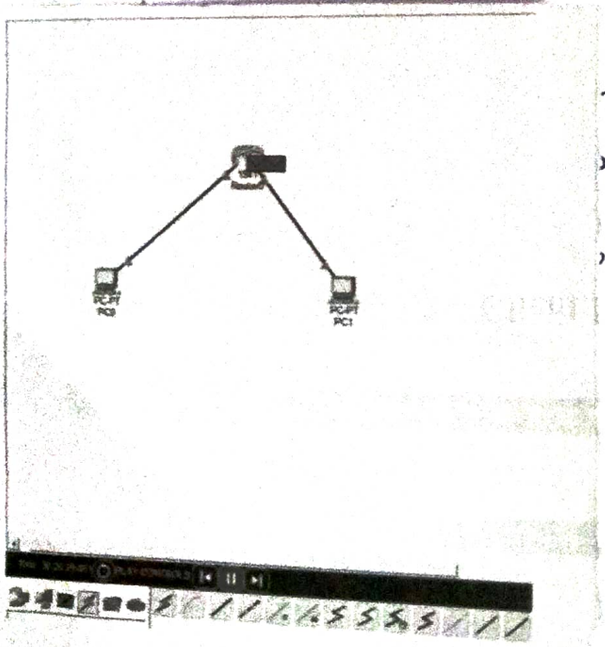
To design & configure and internetwork using a wireless router DHCP server & Internet cloud

Algorithm:

* place wireless router, server, PC & Internet cloud
* network & DHCP service
* DNS setting
* IP address automatically
* PING command



* On packet tracer
* Enable DHCP & configure wireless router
* configure server with IP & DNS setting
* wireless network



...em

...ty using ping

...ing ping cisco.com

The wireless internetwork was configured & DHCP network connectivity were tested

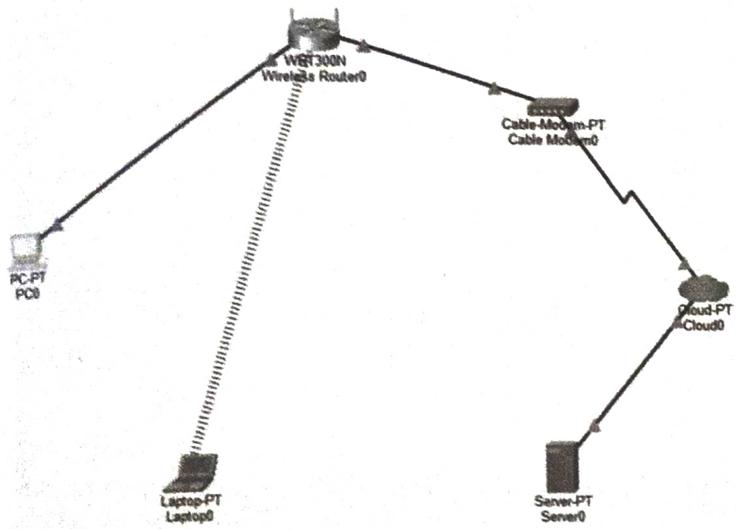
10 (b) Wireless Router, DHCP Server & Internet cloud

Aim:

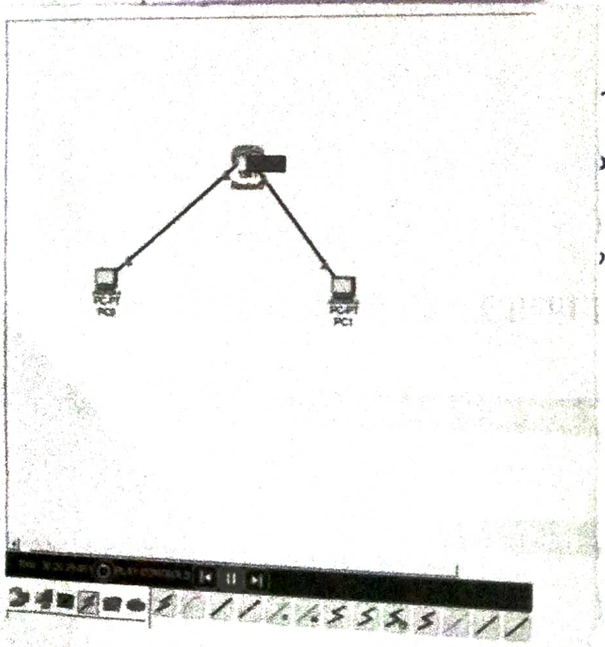
To design & configure and internetwork using a wireless router DHCP server & Internet cloud

Algorithm:

* place wireless router, server, PC & Internet cloud
* network & DHCP service
* DNS setting
* IP address automatically
* PING command



* On packet tracer
* Enable DHCP & configure wireless router
* configure server with IP & DNS setting
* wireless network



...em

...ty using ping

...ing ping cisco.com

The wireless internetwork was configured & DHCP network connectivity were tested

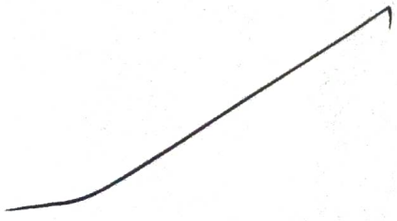
Exp. 12 Routing information protocol
13/10/26

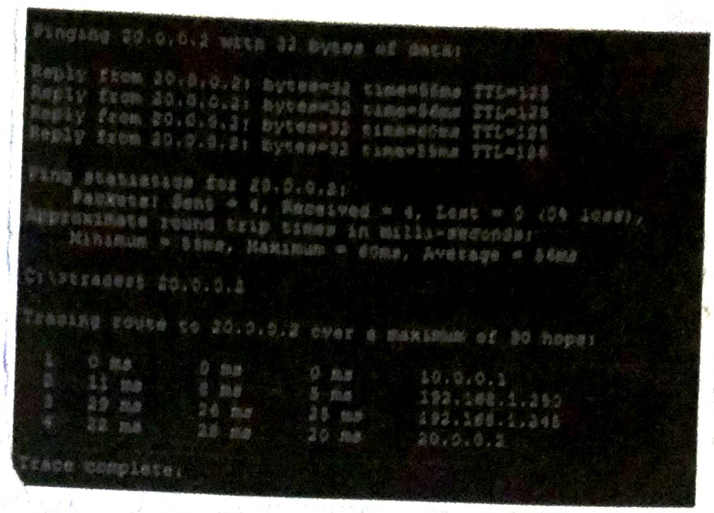
Aim:

To simulate a static routing configuration using cisco packet Tracer

Procedure:

* Open cisco packet tracer and create the given network topology
* Connect a 3 routers and pc's using suitable cables
* Configure Ip address for routers and pc's
* Enable all router interface using no shut down
* Configure static routes on router 0, router 1 and router 2 using ip route.
* Set main and backup router using different administrative distance value
* Test connectivity
* use trace to verify the routing path.
* Thus a static routing is successfully configured and verified






```
Router>enable
Router#show ip route static
S     30.0.0.0/8 [20/0] via 40.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2
S     60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S     30.0.0.0/8 [10/0] via 20.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2
S     60.0.0.0/8 [1/0] via 40.0.0.2
```

Result:

Routing information protocol successfully configure and dynamic routing between PC0 and PC1 was verified.

Exp-13 Implement Your own ping Program
13/10/21

Aim:

To write a python program to implement the ping utility and check network connectivity between two hosts

Algorithm:

* Get destination IP address from user
* Send an ICMP echo request using the system ping command
* Receive the response from the destination
* Display whether or not host is reachable
* Stop the program

Code:

```
import subprocess
import platform
ip = input ("Enter destination IP address : ")
if platform.system().lower() == "windows":
    command = ["ping", "-n", "4", ip]
else:
    command = ["ping", "-c", "4", ip]
result = subprocess.run(command, capture_output=True,
                        text=True)
print(result.stdout)
if result.returncode == 0:
    print("Host is reachable")
else:
    print("Host is unreachable")
```

output :

I/P

Enter destination IP address

8.8.8.8

O/P

Reply from : 8.8.8.8

Host is reachable

Result :

Thus, the python program to check network connectivity using ping utility was executed successfully

Exp-14 Packet Sniffing using Raw Sockets.
13/10/24

Aim:

To write python program using sockets to capture network packets and display their source IP destination IP and protocol

Algorithm:

* create raw socket
* capture incoming IP packet
* Extract IP header from each packet
* identify protocol used
* display packet information

code:

```
import socket
import struct

try:
    sniffer = socket.socket
        socket.AF_INET,
        socket.SOCK_RAW,
        socket.IPPROTO_IP

except OSError as e:
    print("Socket creation failed:", e)
    print("Run with administrator root privileges!")
    raise SystemExit

print("Packet sniffer started. Press Ctrl+C to stop")

try:
    while True:
        packet, address = sniffer.recvfrom(65535)
    if len(packet) < 20:
        continue
```

```
version_ihl = packet[0]
version = version_ihl >> 4
ihl = (version_ihl & 0x0F) * 4
if version != 4 or len(packet) < ihl:
    continue
ttl = packet[8]
protocol = packet[9]
src_ip = socket.inet_ntoa(packet[12:16])
dst_ip = socket.inet_ntoa(packet[16:20])
protocols = {1: "ICMP",
             6: "TCP",
             17: "UDP"}
print("\n packet captured")
print("source ip:", src_ip)
print("Destination IP:", dst_ip)
print("protocol:", protocols.get(protocol, str(protocol)))
print("TTL:", ttl)
print("packet size:", len(packet))
except KeyboardInterrupt:
    print("\n packet sniffing stopped.")
finally:
    sniffer.close()
```

Output:

Packet sniffer started.

Press ctrl + c to stop

Packet captured

Source IP : 192.168.1.10

Destination IP : 192.168.1.1

Protocol : TCP

TTL : 64

Packetsize : 60

Result:

Thus, the python program using raw sockets was implemented to capture network packets executed successfully.